STM32 EMBEDDED FIRMWARE MANUAL

# Smart Cane Assistive System

Complete Program Architecture, Subsystems & Function Reference

TARGET MICROCONTROLLER
**STM32F103C8T6 (ARM Cortex-M3 @ 8 MHz HSI)**

PRIMARY PERIPHERALS
**HC-SR04, NEO-6M GPS, HC-05 BT, TIM2, SysTick**

TOTAL FUNCTIONS CATALOGED
**46 Distinct Functions Across 9 C Modules**

FIRMWARE ARCHITECTURE
**Bare-Metal Superloop + 1ms SysTick Timebase**

TOOLCHAIN & IDE
**STM32CubeIDE / GNU ARM Embedded Toolchain**

DOCUMENT DATE & STATUS
**November 2026 • Production Release**

**Executive Summary:** The Smart Cane firmware is a high-reliability, real-time embedded solution designed to aid visually impaired individuals with spatial awareness, collision avoidance, and personal safety. The system performs ultrasonic obstacle detection, satellite telemetry capture with an anti-glitch jump filter, Bluetooth serial telemetry streaming, and a dual-mode emergency SOS state machine with a 5-second false-alarm cancel window. This document details every function, algorithm, hardware register interaction, and calling relationship within the software.

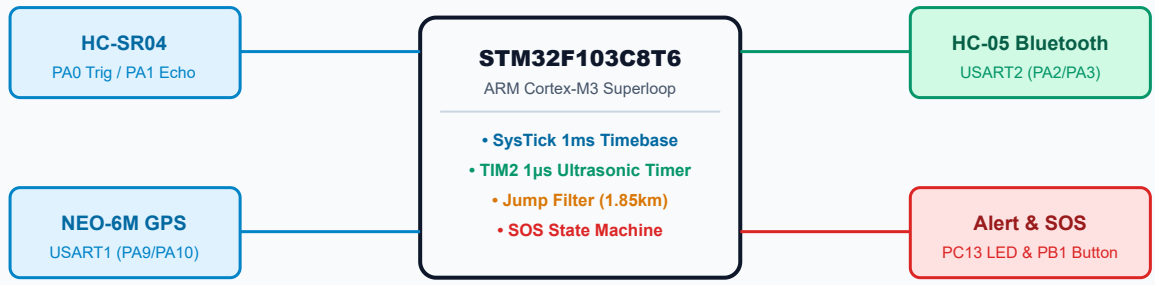
# 1. System Architecture & Hardware Pinout

The system firmware runs directly on the ARM Cortex-M3 silicon without RTOS scheduling jitter, using direct hardware register access for microsecond pulse measurement and non-blocking state machines for serial processing.

## 1.1 Microcontroller Pin Configuration

| SIGNAL / PIN | PERIPHERAL | HARDWARE MODE | OPERATIONAL ROLE |
|---|---|---|---|
| `PA0` | HC-SR04 Trig | GPIO Output Push-Pull (2 MHz) | Issues 20 µs trigger pulse every 100 ms to start distance measurement |
| `PA1` | HC-SR04 Echo | GPIO Input Floating | Receives ultrasonic echo pulse width; timed by TIM2 with 1 µs resolution |
| `PA2` | HC-05 BT TX | USART2 TX (AF Push-Pull, 50 MHz) | Transmits telemetry, warnings, and GPS data at 9600 baud (8-N-1) |
| `PA3` | HC-05 BT RX | USART2 RX (Input Pull-Up) | Receives configuration or handshakes from paired smartphone/receiver |
| `PA9` | NEO-6M GPS TX | USART1 TX (AF Push-Pull, 50 MHz) | Sends NMEA configuration commands to GPS receiver (9600 baud) |
| `PA10` | NEO-6M GPS RX | USART1 RX (Input Pull-Up) | Receives streaming NMEA-0183 sentences ($GPRMC/$GNRMC) at 9600 baud |
| `PB1` | SOS Button | GPIO Input with Internal Pull-Up | Emergency input line; supports momentary push buttons or toggle switches |
| `PC13` | Status LED | GPIO Output Push-Pull (2 MHz) | Active-LOW indicator: solid ON during obstacle proximity; 128 ms strobe during SOS |

## 1.2 System Control & Dataflow Diagram



# 2. Core Application Functions (Src/main.c & Inc/main.h)

The core application functions manage real-time hardware execution, timing coordination, sensor algorithms, and wireless communications.

### `main`

MAIN ENTRY | MAIN.C:614

```
int main(void)
```

Primary program entry point. Initializes MCU hardware registers via `System_Init()` and enters the infinite non-blocking superloop to execute all sub-tasks in round-robin fashion.

| | |
|---|---|
| Calling Pattern: | Invoked by CMSIS startup code `Reset_Handler` after stack and data/BSS initialization. |
| Task Sequence: | 1. `Process_Emergency_Button()` : Checks for PB1 button press or release.<br>2. **GPS Serial Polling:** Tests `USART1->SR & (1 << 5)` (RXNE flag). If a byte arrived, extracts it from `USART1->DR` and calls `Process_GPS_Byte()` .<br>3. `Process_SysTick()` : Checks 1ms tick flag, updates countdowns and flash rates.<br>4. `Process_Ultrasonic()` : Triggers HC-SR04 pulse and reads echo return. |
| Concurrency: | 100% non-blocking cooperative execution. |

### `System_Init`

INIT ENGINE | MAIN.C:61

```
void System_Init(void)
```

Performs bare-metal register-level initialization of clocks, GPIO pins, SysTick, TIM2, and dual USART peripherals for maximum boot speed and zero HAL runtime overhead.

| | |
|---|---|
| Hardware Setup: | • **RCC Clocks:** Powers APB2 (AFIO, GPIOA, GPIOB, GPIOC, USART1) and APB1 (TIM2, USART2).<br>• **GPIOA Configuration:** PA0=Output PP 2MHz, PA1=Input Floating, PA2=AF-PP (USART2 TX), PA3=Input Pull-Up (USART2 RX), PA9=AF-PP (USART1 TX), PA10=Input Pull-Up (USART1 RX).<br>• **GPIOB Configuration:** PB1 set to Input Pull-up mode (ODR=1).<br>• **GPIOC Configuration:** PC13 set to Output PP 2MHz, set HIGH (active-low LED initially OFF).<br>• **SysTick:** LOAD=7999 (1ms @ 8MHz), VAL=0, CTRL=5 (CPU clock, enabled).<br>• **TIM2:** PSC=7 (1µs tick @ 8MHz), ARR=0xFFFF, EGR=1 (prescaler reload).<br>• **USART1 & 2:** Baud rate divisor BRR=0x0341 (9600 baud @ 8MHz), CR1=0x200C (UE, TE, RE).<br>• **Banner:** Transmits welcome banner over Bluetooth upon completion. |
| Registers Modified: | `RCC->APB1ENR` , `RCC->APB2ENR` , `GPIOA/B/C->CRL/CRH/BSRR` , `SysTick->*` , `TIM2->*` , `USART1/2->*` |

### Process_Ultrasonic

PROXIMITY SENSOR | MAIN.C:530

```
void Process_Ultrasonic(void)
```

Non-blocking 100ms trigger and echo measurement engine for the HC-SR04 sensor. Accurately determines distance in centimeters using TIM2 microsecond counting and triggers proximity warnings.

**Algorithm:**

1. **Safety Interlock:** Halts immediately if `emergency_state != EMERGENCY_STATE_NORMAL`.
2. **Periodic Gate:** Executes only when `ultrasonic_timer ≥ 100 ms`.
3. **Trigger Generation:** Pulses PA0 HIGH for 20 µs using calibrated NOP loop, then brings PA0 LOW.
4. **Echo Start Wait:** Polls PA1 until HIGH (timeout: 35,000 iterations ≈ 25 ms).
5. **TIM2 Microsecond Timing:** Resets counter to 0, enables CEN=1, and waits for PA1 to fall LOW (timeout: 45,000 µs). Stops TIM2 and stores count.
6. **Glitch Suppression:** Discards echo pulses <116 µs (<2 cm blind zone).
7. **Distance Math:** Calculates distance in cm: `(raw_echo_us + 29) / 58` (rounded to nearest integer).
8. **Alert Logic:** If distance is between 2 cm and 50 cm: illuminates PC13 LED; if `alert_cooldown ≥ 500 ms`, emits `[ALERT] Obstacle: XX cm` over Bluetooth.

**Key Parameters:** Min: 2 cm, Max: 50 cm, Cooldown: 500 ms, Interval: 100 ms.

### Process_Emergency_Button

SAFETY STATE MACHINE | MAIN.C:353

```
void Process_Emergency_Button(void)
```

Universal debounce and state machine engine that handles both momentary tactile buttons and physical latching toggle switches on PB1. Implements a 5-second cancel window for false alarms.

**State Transitions:**

- **Debounce:** Rejects contact bounce with a 50 ms lockout timer.
- **Normal → Pending SOS:** First press detected: transmits `STR_SOS_BANNER`, sets state to `PENDING`, arms 5-second countdown, and triggers immediate GPS burst.
- **Pending → Normal (False Alarm):** If button is pressed again within 5 seconds, cancels emergency, sends `STR_FALSE_ALARM`, and resets state.
- **Confirmed → Normal (Cleared):** If button is pressed after emergency is active, resets state to Normal, transmits `STR_SOS_CLEARED`, and resumes ultrasonic sensing.
- **Latching Switch Discrimination:** Ignores finger release duration <350 ms. If a switch remained active >350 ms and unlatches, treats transition as an intentional second action.

### Process_GPS_Byte

GPS STREAM | MAIN.C:432

```
void Process_GPS_Byte(uint8_t byte)
```

Serial byte-stream consumer for incoming USART1 data from the NEO-6M module. Detects NMEA sentences, manages buffer boundaries, and triggers sentence parsing.

**Stream States:**

- `GPS_STATE_IDLE`: Ignores incoming bytes to conserve MCU cycles between 5-second trigger intervals.
- `GPS_STATE_WAIT_DOLLAR`: Waits for '$' framing start-of-sentence marker.
- `GPS_STATE_BUFFERING`: Collects characters into `line_buf[128]`. Protects against buffer overrun. Upon ' ' arrival, calls `Parse_RMC()`.

### Parse_RMC

NMEA TOKENIZER | MAIN.C:178

```
bool Parse_RMC(char *line)
```

High-performance zero-copy NMEA parser. Validates sentence headers and extracts GPS satellite fix status, latitude, north/south indicator, longitude, and east/west indicator.

| | |
|---|---|
| Validation Logic: | Validates `line[0] == '$'` and `line[3..5] == 'RMC'` (matching both `$GPRMC` and `$GNRMC`). Scans comma delimiters: Field 2 = Fix Status ('A' = Valid, 'V' = Warning), Field 3 = Latitude, Field 4 = N/S, Field 5 = Longitude, Field 6 = E/W. Terminates string fields in-place with null characters. |
| Returns: | `true` if sentence contains valid RMC fields with ≥6 commas; `false` otherwise. |

### Filter_And_Send_GPS

ANTI-GLITCH FILTER | MAIN.C:288

```
void Filter_And_Send_GPS(void)
```

Kinematic sanity filter that prevents transmission of corrupted coordinates resulting from satellite multipath, poor signal, or UART transmission noise.

| | |
|---|---|
| Filtering Criteria: | 1. Converts latitude and longitude into total tenths of an arcminute.<br>2. Compares against previous baseline reading: if coordinate jump exceeds 10 tenths (≈ 1.85 km in 5 seconds) or hemisphere character flips ('N'→'S' or 'E'→'W'), flags reading as a glitch.<br>3. Overrides erroneous fix with previous known-good coordinates and transmits clean data.<br>4. If reading is valid, updates baseline coordinates. |

### Send_Parsed_GPS

TELEMETRY FORMAT | MAIN.C:235

```
void Send_Parsed_GPS(void)
```

Formats and transmits human-readable GPS coordinates over USART2 (Bluetooth). Prepends emergency alert headers if emergency SOS is active, or outputs a searching notification if satellite lock is pending.

| | |
|---|---|
| Output Format: | • Lock Available: `Lat: 1258.1234 N, Lon: 07735.5678 E`<br>• No Lock: `GPS: Searching for Satellites... (No Lock)`<br>• SOS Active: Prepends `*** [EMERGENCY GPS LOCATION] ***` |

### Compute_Lat_Tenths & Compute_Lon_Tenths

COORDINATE MATH | MAIN.C:263, 275

```
static uint32_t Compute_Lat_Tenths(const char *str)
static uint32_t Compute_Lon_Tenths(const char *str)
```

High-speed integer parsing routines. Converts NMEA coordinate strings ( `DDMM.M...` for latitude, `DDDMM.M...` for longitude) directly into tenths of a minute: `(Degrees × 60 + Minutes) × 10 + Tenths`.

| | |
|---|---|
| Advantage: | Eliminates floating-point library overhead; executes in <1 µs. |

### Process_SysTick

TIMEBASE MAIN.C:478

```
void Process_SysTick(void)
```

Hardware SysTick polling engine. Monitors bit 16 (COUNTFLAG). Every 1 ms, advances system counters, drives the emergency countdown, operates the SOS LED strobe, and triggers periodic GPS capture.

Key Actions:

- Increments `ms_counter`, `gps_timer`, `ultrasonic_timer`, `alert_cooldown`.
- Decrements `btn_debounce_timer`.
- **5-Second SOS Confirmation:** If `emergency_state == PENDING` and timer reaches 5000 ms, promotes state to `CONFIRMED` and transmits `STR_SOS_CONFIRMED`.
- **LED Strobe:** When emergency is active, toggles PC13 at ~128 ms intervals (tested via bit 7 of `ms_counter`).
- **5-Second GPS Trigger:** When `gps_timer ≥ 5000 ms`, resets timer and arms GPS receiver to capture next sentence.

### UART2_SendChar, UART2_SendString, UART2_SendNum, UART2_Send2DigitNum

UART DRIVERS MAIN.C:131, 141, 150, 164

```
void UART2_SendChar(char c)
void UART2_SendString(const char *str)
void UART2_SendNum(uint32_t num)
void UART2_Send2DigitNum(uint32_t num)
```

Low-latency serial transmission library. Polls TXE (bit 7 of `USART2->SR`) and writes to `USART2->DR`. Formats decimal numbers (0-999) without calling heavy `sprintf` or dynamic memory allocators.

### Error_Handler

SAFETY TRAP MAIN.C:606

```
void Error_Handler(void)
```

Global exception trap. Disables global interrupts via `__disable_irq()` and traps CPU in an infinite loop if peripheral hardware initialization fails.

# 3. Hardware Abstraction Layer Modules (Drivers)

STM32Cube HAL driver functions that provide peripheral clock gating, alternate function multiplexing, and low-level peripheral configuration.

## 3.1 GPIO Initialization Module (Src/gpio.c)

### MX_GPIO_Init

HAL INIT | GPIO.C:42

```
void MX_GPIO_Init(void)
```

Enables peripheral clock gates for GPIOC, GPIOD, GPIOA, and GPIOB via RCC bus registers.

## 3.2 I2C Peripheral Driver (Src/i2c.c)

### MX_I2C1_Init

HAL DRIVER | I2C.C:30

```
void MX_I2C1_Init(void)
```

Configures I2C1 instance for 100 kHz standard mode, 7-bit addressing, duty cycle 2, general call disabled, and no-stretch mode disabled. Traps to `Error_Handler()` on error.

### HAL_I2C_MspInit & HAL_I2C_MspDeInit

MSP CALLBACK | I2C.C:59, 87

```
void HAL_I2C_MspInit(I2C_HandleTypeDef* i2cHandle)
void HAL_I2C_MspDeInit(I2C_HandleTypeDef* i2cHandle)
```

Low-level MSP hardware callbacks for I2C1. Configures PB6 (SCL) and PB7 (SDA) as alternate function open-drain, enables/disables I2C1 peripheral clock in RCC.

## 3.3 Timer 2 Peripheral Driver (Src/tim.c)

### MX_TIM2_Init

HAL DRIVER | TIM.C:30

```
void MX_TIM2_Init(void)
```

Configures TIM2 general purpose timer instance, timebase clock division, and master mode trigger output (TRGO). Traps to `Error_Handler()` on error.

### HAL_TIM_Base_MspInit & HAL_TIM_Base_MspDeInit

MSP CALLBACK | TIM.C:70, 86

```
void HAL_TIM_Base_MspInit(TIM_HandleTypeDef* tim_baseHandle)
void HAL_TIM_Base_MspDeInit(TIM_HandleTypeDef* tim_baseHandle)
```

Enables or disables TIM2 peripheral clock gate in `RCC->APB1ENR`.

## 3.4 Serial USART Drivers (Src/usart.c)

### MX_USART1_UART_Init & MX_USART2_UART_Init

HAL DRIVER USART.C:32, 61

```
void MX_USART1_UART_Init(void)
void MX_USART2_UART_Init(void)
```

Initializes USART1 (GPS) and USART2 (Bluetooth) instances for 9600 baud, 8-N-1, TX/RX mode, and 16x oversampling. Traps to `Error_Handler()` on error.

### HAL_UART_MspInit & HAL_UART_MspDeInit

MSP CALLBACK USART.C:89, 155

```
void HAL_UART_MspInit(UART_HandleTypeDef* uartHandle)
void HAL_UART_MspDeInit(UART_HandleTypeDef* uartHandle)
```

Configures GPIO pins (PA9/PA10 for USART1, PA2/PA3 for USART2), enables peripheral clock gates, and registers NVIC interrupt vectors with priority 0.

## 3.5 Global MCU Support Package (Src/stm32f1xx_hal_msp.c)

### HAL_MspInit

MSP CORE STM32F1XX_HAL_MSP.C:62

```
void HAL_MspInit(void)
```

Global MCU Support Package initialization. Enables AFIO clock in RCC and configures NVIC priority grouping to `NVIC_PRIORITYGROUP_4` (4 bits preemption, 0 bits subpriority).

# 4. Interrupt Service Routines & Cortex-M3 Faults (Src/stm32f1xx_it.c)

Low-level vector interrupt handlers and processor fault handlers for Cortex-M3 core operations and peripheral events.

## 4.1 Processor Fault & Exception Traps

### NMI_Handler

ISR STM32F1XX_IT.C:70

```
void NMI_Handler(void)
```

Non-Maskable Interrupt handler. Traps execution in an infinite loop upon critical hardware signals.

### HardFault_Handler

FATAL FAULT STM32F1XX_IT.C:85

```
void HardFault_Handler(void)
```

Fatal hardware fault trap. Catches unrecoverable errors (invalid vector fetch, escalated faults, bus errors) in an infinite loop.

### MemManage_Handler

MPU FAULT STM32F1XX_IT.C:100

```
void MemManage_Handler(void)
```

Memory Protection Unit (MPU) fault trap. Halts CPU upon memory access rule violations.

### BusFault_Handler

BUS FAULT STM32F1XX_IT.C:115

```
void BusFault_Handler(void)
```

Memory bus fault handler. Traps data read/write bus transfer errors and pre-fetch aborts.

### UsageFault_Handler

CPU FAULT STM32F1XX_IT.C:130

```
void UsageFault_Handler(void)
```

Program fault handler. Catches undefined CPU instructions, unaligned memory access, or divide-by-zero errors.

### SVC_Handler, DebugMon_Handler, PendSV_Handler

SYSTEM ISR | STM32F1XX_IT.C:145, 158, 171

```
void SVC_Handler(void)
void DebugMon_Handler(void)
void PendSV_Handler(void)
```

Cortex-M3 supervisor call, debug monitor, and pending software interrupt handlers reserved for system operations.

## 4.2 Peripheral Vector Interrupts

### SysTick_Handler

TIMER ISR | STM32F1XX_IT.C:184

```
void SysTick_Handler(void)
```

Executes every 1 millisecond on SysTick exception. Calls `HAL_IncTick()` to maintain HAL tick timing.

### USART1_IRQHandler & USART2_IRQHandler

UART ISR | STM32F1XX_IT.C:205, 219

```
void USART1_IRQHandler(void)
void USART2_IRQHandler(void)
```

Dispatches hardware interrupts for USART1 (GPS) and USART2 (Bluetooth) to their respective HAL handlers.

# 5. System Clock, CMSIS & POSIX System Calls

## 5.1 System Clock Engine (Src/system_stm32f1xx.c)

### SystemInit

CMSIS SYSTEM_STM32F1XX.C:175

```
void SystemInit(void)
```

Resets RCC configuration to default state, enables internal HSI oscillator, disables PLL and clock interrupts, and configures the Vector Table Base address.

### SystemCoreClockUpdate

CMSIS SYSTEM_STM32F1XX.C:224

```
void SystemCoreClockUpdate(void)
```

Dynamically recalculates the global `SystemCoreClock` variable (8000000 Hz) based on RCC clock register values.

### SystemInit_ExtMemCtl

CMSIS SYSTEM_STM32F1XX.C:349

```
void SystemInit_ExtMemCtl(void)
```

FSMC external memory controller setup stub (unused on medium-density STM32F103C8T6 devices).

## 5.2 Dynamic Memory Management (Src/sysmem.c)

### _sbrk

HEAP SYSMEM.C:53

```
void *_sbrk(ptrdiff_t incr)
```

Low-level memory allocation engine for C runtime. Extends heap boundary while guarding against collision with the stack pointer (stack overflow guard). Returns `(void*)-1` on out-of-memory.

## 5.3 POSIX C-Runtime System Calls (Src/syscalls.c)

Implements standard Newlib C library hooks for embedded bare-metal execution:

| SYSTEM CALL FUNCTION | LINE | FUNCTIONAL BEHAVIOR ON EMBEDDED TARGET |
|---|---|---|
| `__io_putchar(int ch)` | 35 | Weak symbol redirected to serial UART for character output. |
| `__io_getchar(void)` | 36 | Weak symbol redirected to serial UART for character input. |
| `initialise_monitor_handles(void)` | 44 | Initializes semihosting debug stream handles. |
| `_read(int file, char *ptr, int len)` | 67 | Reads characters using `__io_getchar()`. |
| `_write(int file, char *ptr, int len)` | 80 | Writes characters using `__io_putchar()`. |

| SYSTEM CALL FUNCTION | LINE | FUNCTIONAL BEHAVIOR ON EMBEDDED TARGET |
|---|---|---|
| `_close(int file)` | 92 | Closes file descriptor; returns -1 (no filesystem). |
| `_fstat(int file, struct stat *st)` | 99 | Populates file status; marks device as character-mode. |
| `_isatty(int file)` | 106 | Returns 1 (indicates output stream is a TTY terminal). |
| `_lseek(int file, int ptr, int dir)` | 112 | Repositions file pointer; returns 0. |
| `_open(char *path, int flags, ...)` | 120 | Opens named file; returns -1 (no file system present). |
| `_wait(int *status)` | 128 | Waits for child process to terminate; returns -1. |
| `_unlink(char *name)` | 135 | Removes file entry; returns -1. |
| `_times(struct tms *buf)` | 142 | Returns elapsed process execution time; returns -1. |
| `_stat(char *file, struct stat *st)` | 148 | Returns status of a file; returns 0. |
| `_link(char *old, char *new)` | 155 | Creates file link; returns -1. |
| `_fork(void)` | 163 | Forks process; returns -1 (unsupported on bare metal). |
| `_execve(...)` | 169 | Replaces process image; returns -1. |
| `_getpid(void)` | 48 | Returns process identifier (constant 1). |
| `_kill(int pid, int sig)` | 53 | Signals process; sets errno=EINVAL and returns -1. |
| `_exit(int status)` | 61 | Terminates program execution by locking in an infinite loop. |

# 6. Master Function Cross-Reference Index

Alphabetical directory of all 46 functions across the entire Smart Cane firmware:

| # | FUNCTION NAME | SOURCE MODULE | TYPE / SCOPE | ROLE & RESPONSIBILITY |
|---|---|---|---|---|
| 1 | `BusFault_Handler` | stm32f1xx_it.c | ISR / Trap | Cortex-M3 Bus Fault Trap |
| 2 | `Compute_Lat_Tenths` | main.c | Static Helper | Converts NMEA latitude to arcminute tenths |
| 3 | `Compute_Lon_Tenths` | main.c | Static Helper | Converts NMEA longitude to arcminute tenths |
| 4 | `DebugMon_Handler` | stm32f1xx_it.c | ISR / Trap | Debug Monitor Exception Handler |
| 5 | `Error_Handler` | main.c | Public Trap | Disables interrupts and halts on error |
| 6 | `Filter_And_Send_GPS` | main.c | Public Engine | Rejects coordinate jumps >1.85 km in 5s |
| 7 | `HAL_I2C_MspDeInit` | i2c.c | HAL Callback | Releases I2C1 GPIOs and clock |
| 8 | `HAL_I2C_MspInit` | i2c.c | HAL Callback | Initializes I2C1 PB6/PB7 pins and clock |
| 9 | `HAL_MspInit` | stm32f1xx_hal_msp.c | HAL Core | Initializes AFIO clock & NVIC Priority Group 4 |
| 10 | `HAL_TIM_Base_MspDeInit` | tim.c | HAL Callback | Disables TIM2 clock gate in RCC |
| 11 | `HAL_TIM_Base_MspInit` | tim.c | HAL Callback | Enables TIM2 clock gate in RCC |
| 12 | `HAL_UART_MspDeInit` | usart.c | HAL Callback | Releases USART1/2 GPIOs and NVIC |
| 13 | `HAL_UART_MspInit` | usart.c | HAL Callback | Initializes USART1/2 GPIOs and NVIC |
| 14 | `HardFault_Handler` | stm32f1xx_it.c | ISR / Trap | Fatal Cortex-M3 Hardware Fault Trap |
| 15 | `initialise_monitor_handles` | syscalls.c | Runtime Hook | Semihosting Handle Initializer |
| 16 | `main` | main.c | Core Main | Microcontroller Superloop & Dispatcher |
| 17 | `MemManage_Handler` | stm32f1xx_it.c | ISR / Trap | Memory Protection Unit (MPU) Fault Trap |
| 18 | `MX_GPIO_Init` | gpio.c | HAL Driver | Enables GPIO bus clock gates |
| 19 | `MX_I2C1_Init` | i2c.c | HAL Driver | Configures I2C1 for 100 kHz standard mode |
| 20 | `MX_TIM2_Init` | tim.c | HAL Driver | Configures TIM2 timebase and trigger |
| 21 | `MX_USART1_UART_Init` | usart.c | HAL Driver | Configures USART1 (GPS) for 9600 baud |
| 22 | `MX_USART2_UART_Init` | usart.c | HAL Driver | Configures USART2 (BT) for 9600 baud |
| 23 | `NMI_Handler` | stm32f1xx_it.c | ISR / Trap | Non-Maskable Interrupt Trap |
| 24 | `Parse_RMC` | main.c | Public Parser | Tokenizes $GPRMC/$GNRMC sentences |
| 25 | `PendSV_Handler` | stm32f1xx_it.c | ISR / Trap | Pending Service Call Handler |

| # | FUNCTION NAME | SOURCE MODULE | TYPE / SCOPE | ROLE & RESPONSIBILITY |
|---|---|---|---|---|
| 26 | `Process_Emergency_Button` | main.c | Public Engine | PB1 debounce & 5s cancel state machine |
| 27 | `Process_GPS_Byte` | main.c | Public Engine | Consumes USART1 bytes into sentence buffer |
| 28 | `Process_SysTick` | main.c | Public Engine | 1ms software timebase & LED strobe |
| 29 | `Process_Ultrasonic` | main.c | Public Engine | HC-SR04 pulse measurement & alert trigger |
| 30 | `Send_Parsed_GPS` | main.c | Public Telemetry | Transmits human-readable lat/lon to BT |
| 31 | `SVC_Handler` | stm32f1xx_it.c | ISR / Trap | Supervisor Call Handler |
| 32 | `SysTick_Handler` | stm32f1xx_it.c | ISR / Timebase | 1ms SysTick interrupt; calls HAL_IncTick |
| 33 | `System_Init` | main.c | Public Init | Bare-metal register initialization |
| 34 | `SystemCoreClockUpdate` | system_stm32f1xx.c | CMSIS API | Recalculates CPU clock frequency |
| 35 | `SystemInit` | system_stm32f1xx.c | CMSIS API | Hardware reset state RCC & vector table |
| 36 | `SystemInit_ExtMemCtl` | system_stm32f1xx.c | CMSIS API | External memory bus controller stub |
| 37 | `UART2_Send2DigitNum` | main.c | Public Comm | Transmits 2-digit distance values over BT |
| 38 | `UART2_SendChar` | main.c | Public Comm | Transmits single character over USART2 |
| 39 | `UART2_SendNum` | main.c | Public Comm | Transmits 0-999 decimal integer over BT |
| 40 | `UART2_SendString` | main.c | Public Comm | Transmits null-terminated string over BT |
| 41 | `UsageFault_Handler` | stm32f1xx_it.c | ISR / Trap | Instruction / Alignment CPU Fault Trap |
| 42 | `USART1_IRQHandler` | stm32f1xx_it.c | ISR / Comm | Dispatches USART1 interrupts to HAL |
| 43 | `USART2_IRQHandler` | stm32f1xx_it.c | ISR / Comm | Dispatches USART2 interrupts to HAL |
| 44 | `_sbrk` | sysmem.c | Memory Manager | Heap allocator with stack collision guard |
| 45 | `__io_putchar / __io_getchar` | syscalls.c | POSIX Bridge | Standard I/O character stream hooks |
| 46 | `_read / _write / _exit / etc.` | syscalls.c | POSIX System | Standard C runtime POSIX stubs |